

ПРАКТИЧНИЙ ДОКУМЕНТ ПЕРЕДАЧІ

Передача проєкту e-osvita розробнику

Що потрібно знати, отримати й перевірити, щоб безпечно підтримувати backend, worker, базу даних і адміністративну панель.

Node.js + TypeScript

PostgreSQL + Prisma

Redis + BullMQ

React Admin SPA

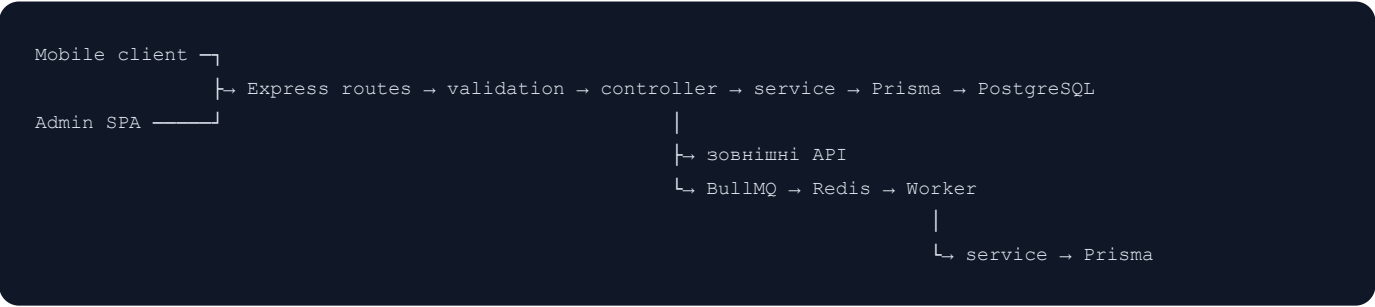
1. Що саме передається

e-osvita — монорепозиторій із трьома окремими runtime-компонентами: Express API, BullMQ worker та React Admin SPA. Проект обслуговує мобільний застосунок, локальну PostgreSQL, зовнішню систему університету, OTP/SMS/email і банківський процесинг.

Головне правило підтримки: API без worker виглядає працездатним, але синхронізації не виконуються. Під час діагностики завжди перевіряйте API, worker, PostgreSQL і Redis окремо.

Компонент	Точка входу	Роль
API	<code>src/index.ts</code> , <code>src/app.ts</code>	HTTP, auth, validation, бізнес-операції
Worker	<code>src/worker.ts</code>	Фонові й періодичні синхронізації
Admin SPA	<code>admin-spa/src/main.tsx</code>	Операційний web-інтерфейс
Database	<code>prisma/schema.prisma</code>	Основне сховище даних
Deployment	<code>Dockerfile</code> , <code>helm/</code>	Збірка і Kubernetes

2. Архітектура в одному рисунку



У проєкті немає окремого repository layer: більшість сервісів звертається до Prisma напряму. Маршрути мобільного API змонтовані на `/v1` , адміністративного — на `/admin/api` , production SPA — на `/admin` .

3. Перший локальний запуск

Вимоги

- Node.js не нижче 20.9;

- PostgreSQL;
- Redis;
- два набори npm dependencies: корінь і `admin-spa` ;
- локальний `.env` .

```
npm install
cd admin-spa && npm install && cd ..

npm run db:generate
npm run db:migrate
```

Запуск у трьох терміналах

```
# 1. API
npm run dev

# 2. Worker
npm run worker:dev

# 3. Admin SPA
cd admin-spa
npm run dev -- --host 127.0.0.1
```

Health	<code>http://127.0.0.1:3000/v1/actuator/health</code>
Swagger	<code>http://127.0.0.1:3000/v1/docs/</code>
Admin dev	<code>http://127.0.0.1:5173/api/admin/</code>

4. Доступи, які треба передати окремо

Секрети не повинні знаходитися в цьому документі або Git. Передача виконується через корпоративний password manager.

Доступ	Навіщо	Що перевірити
GitLab	Код, merge requests, CI/CD	Рівень доступу, protected branches
Config maps / secrets	Environment усіх середовищ	Власник, rotation, різниця dev/test/prod
PostgreSQL	Міграції та діагностика	Окремі read-only і migration roles
Redis	Черги BullMQ	TLS, пароль, namespace, retention
Kubernetes	API/worker deployments	Namespaces, logs, rollback
MAUP API	Навчальні дані	Test/prod URL, allowlist, SLA
OTP/SMS/email	Реєстрація і recovery	Service accounts і quotas
TASLink	Платежі й токенизація	Test terminal, callback URLs
Kafka/Logstash	Централізовані логи	Topics, dashboards, retention

5. Карта коду

Папка	Що шукати
src/routes	URL, HTTP-метод, auth і validation middleware
src/controllers	Перетворення HTTP-запиту у виклик сервісу
src/services	Основна бізнес-логіка та Prisma-запити
src/jobs	Обробники BullMQ
src/config/queue.ts	Оголошення черг та Redis connection
src/middlewares	JWT, validation, errors, XSS, logging
src/validations	Joi-схеми вхідних запитів
src/types	DTO зовнішніх систем і внутрішні типи
prisma	Схема та історія міграцій
admin-spa/src/pages	Сторінки React
admin-spa/src/services	Frontend API clients
docs	Синхронізація, платежі й observability

Для вивчення функції проходите вертикально: `route` → `validation` → `controller` → `service` → `Prisma/job` . Не починайте з читання всього сервісного шару поспіль.

6. Ключові бізнес-потoki

Авторизація

Телефон → OTP → User → password → access/refresh JWT. Refresh token зберігається у БД. Невдалі входи збільшують лічильник і можуть заблокувати login до recovery.

Верифікація студента

Авторизований користувач вибирає організацію → backend запитує `/studentinfo` → створює `UserStudent`, department/group → ставить фонові jobs.

Синхронізація

API додає job у Redis → worker викликає зовнішній API → нормалізує відповідь → змінює дані та `StudentSyncStatus`/`OrganizationSyncStatus`.

Платіж

Створення `StudentPayment` → payment link або token purchase → 3DS → callback/webhook → перевірка стану → `PaymentTranState`.

7. База даних і міграції

Prisma-схема містить 28 моделей. Критичні зв'язки: `User → UserStudent → Department → Organization`, а також `StudentPayment → PaymentTranState`.

Правила змін

1. Не редагувати вже застосовані production-міграції.
2. Створювати нову міграцію і переглядати SQL до застосування.
3. Перевіряти міграцію на чистій БД та копії production-схеми.
4. Планувати rollback або forward fix для незворотних змін.
5. Після зміни схеми запускати `prisma generate`, TypeScript build і tests.

Відоме розходження: Prisma enum містив `SyncStatus.SUSPENDED`, але всі 29 міграцій не створювали це значення у PostgreSQL. У локальній тестовій БД воно було додано вручну; для репозиторію потрібна коректна міграція.

8. Redis, BullMQ і worker

Окремі черги існують для розкладу, екзаменів, оцінок, навчального плану, новин, платежів, знижок і recovery. Worker створює consumer для кожного типу з concurrency 5; janitor має concurrency 1.

Під час проблеми перевірити

1. Чи доступний Redis.
2. Чи запущено worker deployment.
3. Waiting/active/failed jobs і repeatable configuration.
4. Чи не залишився DB status `RUNNING` після падіння.
5. Чи збігаються cron у PostgreSQL та Redis.
6. Чи є job idempotent при повторному запуску.

Транзакція Prisma не охоплює Redis. Можливий стан «БД оновилась, job не створився» або навпаки.
Це потрібно враховувати при recovery.

9. Admin SPA

React 19 SPA використовує React Router, TanStack Query, Zustand, Axios, Chart.js і Tailwind. Серверні дані мають залишатися в TanStack Query, локальний UI state — у Zustand.

Маршрути

- Dashboard;
- Students і Student Inspector;
- Student Invite;
- Jobs;
- Organizations і Organization Detail.

Перевіряйте одночасно три префікси: React Router `basename`, Vite `base` і Express mount. Зараз `/admin`, `/admin/api` та `/api/admin` не повністю уніфіковані. У репозиторії також залишились старі EJS-сторінки.

10. Мінімальні перевірки перед зміною

```
# Backend build
npm run build

# Admin build
cd admin-spa && npm run build

# Runtime
curl http://127.0.0.1:3000/v1/actuator/health
redis-cli ping
psql -d mydb -c "select now();"
```

У `tests/integration` є тести auth, university, processing і transliteration. Штатний `npm test` зараз потребує виправлення: він викликає відсутній script `db:push`. Не вважайте зелену TypeScript-збірку доказом працездатності міграцій або інтеграцій.

Обов'язкові набори тестів, яких бракує

- authorization для кожного admin endpoint;
- tenant isolation;
- чиста послідовність усіх міграцій;
- contract tests відповідей університету;
- idempotency і retry jobs;
- callback/webhook security;
- end-to-end login, student verification і sync.

11. Відомий технічний борг і ризики

Пріоритет	Проблема	Наслідок
Critical	Неоднакова auth-перевірка admin routes	Несанкціоновані операції
Critical	Schema/migration drift	Runtime-помилки на чистій БД
High	TLS verification MAUP вимкнено	Ризик MITM
High	ADMIN не обмежений organizationId	Немає tenant-ізоляції
High	Немає історії sync runs та audit log	Складне розслідування
High	Немає атомарності DB + queue	Розходження станів
Medium	Стара EJS і нова SPA паралельно	Дублювання й різна поведінка
Medium	Історичні помилки назв полів	Складна інтеграція та міграції
Medium	npm dependency vulnerabilities	Потрібен контрольований upgrade

12. План перших п'яти днів

День	Результат
1	Отримати доступи, запустити API/worker/SPA, пройти health і login
2	Пройти route → controller → service → Prisma для студентського профілю
3	Вручну простежити один job від Redis до запису в PostgreSQL
4	Перевірити платежі в test environment без реальних коштів
5	Зафіксувати environment matrix, runbook, ризики й першу безпечну задачу

Добра перша задача: виправити test script, додати migration test і закрити auth на одному наборі admin endpoints. Не починати передачу з великого рефакторингу sync engine.

13. Релізний чек-лист

1. Зміни переглянуті іншим розробником.
2. Backend і admin SPA збираються.
3. Tests і migration test зелені.
4. Нові environment variables описані без секретів.
5. Міграція перевірена й має план відновлення.
6. Не змінено callback URLs або queue names без узгодження.
7. Оновлені версії package та Helm chart.
8. API і worker розгортаються як окремі workloads.
9. Після deployment перевірені health, login, Redis і контрольний job.
10. Відома команда rollback і відповідальна особа.

14. Короткий runbook інциденту

API не відповідає

Перевірити pod/process, PostgreSQL connection, останню міграцію, config validation і application logs.

Дані не оновлюються

Перевірити worker, Redis, failed jobs, зовнішній API, credentials, `StudentSyncStatus`, stuck `RUNNING` і cron reconciliation.

Помилки лише одного студента

Перевірити externalId, статус студента, department/organization, останню помилку кожного sync type. Не виконувати віре без резервного плану.

Платіж завис

Знайти `paymentRef`, `sid`, локальний status і `PaymentTranState`; звірити test/prod terminal та callback. Не змінювати фінальний статус SQL-командою без погодженої процедури.

Після релізу зросли помилки

Зупинити подальший rollout, порівняти API і worker versions, перевірити міграції, queue payload compatibility та виконати rollback.

15. Акт передачі

Пункт	Статус / коментар
Репозиторій і branches передані	_____
Environment matrix передано	_____
Secrets передані через захищений канал	_____
Локальний запуск виконано разом	_____
Deployment і rollback продемонстровано	_____
Один sync job простежено	_____
Test payment flow продемонстровано	_____
Dashboards і логи доступні	_____
Відкриті інциденти та борги зафіксовані	_____
Контакти команд і SLA передані	_____

Передав: _____ Прийняв: _____

Дата: _____ Версія/commit: _____

Критерій успішної передачі: новий розробник самостійно запускає систему, знаходить потрібний код, виконує безпечну зміну, проводить її через перевірки та може діагностувати зупинену синхронізацію без усних підказок.